

ARTIFICIAL INTELLIGENCE

Assignment – 1

AI Smart Energy Management System

An Intelligent Agent-Based Approach to Energy Efficiency

Submitted By

Devi Parvathy K

Group 8

S7 CSE-A

Roll No: 25

1. Introduction

The AI Smart Energy Management System is an intelligent agent designed to minimize electricity consumption in buildings while maintaining occupant comfort. The system monitors factors such as room occupancy, indoor temperature, light level, and outdoor temperature to make intelligent decisions regarding HVAC systems and lighting.

The intelligent agent automatically controls energy-consuming devices based on predefined comfort requirements and current environmental conditions. It reduces unnecessary energy consumption by switching off HVAC and lighting in unoccupied rooms and adjusting artificial lighting according to available daylight.

2. PEAS Representation

Agent Type	Performance Measure	Environment	Actuators	Sensors
AI Smart Energy Management System	Minimize electricity consumption Maintain occupant comfort Reduce unnecessary energy usage Improve energy efficiency Maximize energy savings	Smart building rooms Occupants HVAC systems Lighting systems	HVAC controller Smart lights / dimmer Smart plugs	Occupancy sensor Indoor temperature sensor Light sensor Outdoor temperature feed Appliance power meter

Table 1: PEAS Representation of the AI Smart Energy Management System

3. Environment Properties

Property	Type	Justification
Observability	Partially Observable	Sensor readings contain noise, and the agent cannot know every environmental factor exactly.
Determinism	Stochastic	Occupancy, temperature changes, and environmental conditions involve random variations.
Dynamics	Dynamic	Room temperature, occupancy, outdoor temperature, and daylight change over time.
Discreteness	Continuous	Temperature, light level, and energy consumption can take continuous values.
Agents	Single Agent	A single intelligent agent controls the energy-management decisions for all rooms.
Knowledge	Partially Known	The agent knows comfort requirements and control rules, but environmental changes are not completely predictable.

Table 2: Environment Properties of the AI Smart Energy Management System

4. Rationality Measure

A rational agent selects actions that maximize its performance based on the information available.

In the proposed AI Smart Energy Management System, the agent aims to minimize electricity consumption while maintaining occupant comfort. It checks room occupancy and temperature before deciding whether HVAC should be activated. HVAC is switched off when a room is unoccupied, thereby preventing unnecessary energy consumption.

The agent also uses daylight harvesting for lighting. When sufficient natural light is available, artificial lighting is reduced or switched off. When the room is occupied and the light level is low, the system increases the artificial lighting level.

The agent also uses a comfort band around the preferred temperature. This prevents the system from continuously operating the HVAC when the room temperature is already within an acceptable range.

Therefore, the system behaves rationally by selecting actions that provide a suitable balance between energy savings and occupant comfort.

5. Innovation Points

1. Occupancy-Based Energy Control

The system detects whether a room is occupied and automatically switches off unnecessary HVAC and lighting when the room is empty.

2. Daylight Harvesting

The system measures the available natural light and automatically adjusts the brightness of artificial lights. This reduces electricity consumption while maintaining adequate lighting.

3. Predictive / Pre-emptive HVAC Control

The system considers the current temperature and comfort limits and activates HVAC before the room moves significantly outside the comfort range. This helps maintain comfort without continuously running the HVAC system.

6. System Architecture Diagram

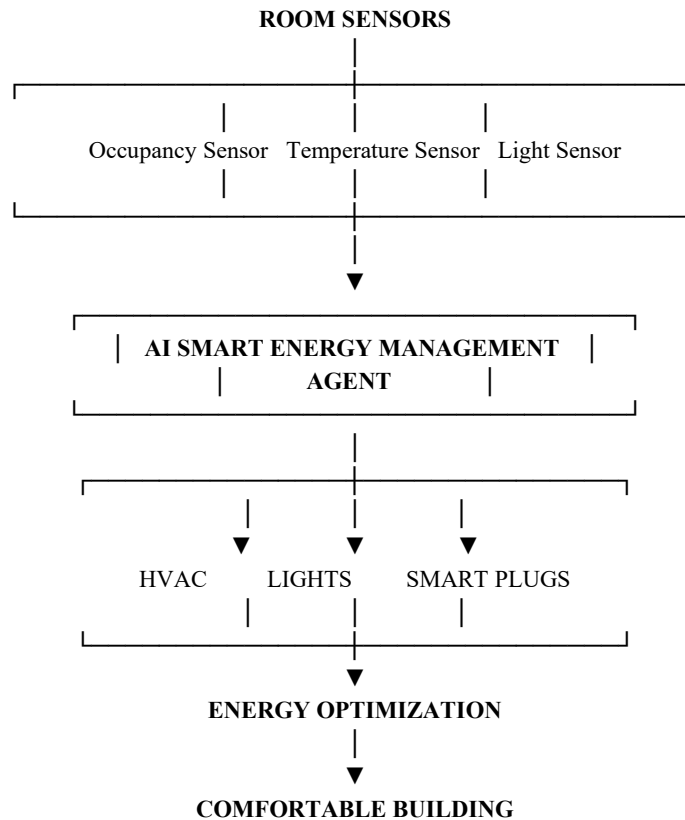


Figure 1: Architecture of the AI Smart Energy Management System

7. Conclusion

The AI Smart Energy Management System provides an intelligent solution for reducing electricity consumption in buildings while maintaining occupant comfort. By monitoring occupancy, temperature, and light levels, the intelligent agent makes appropriate decisions regarding HVAC and lighting operation.

The system reduces unnecessary energy usage through occupancy-based control, daylight harvesting, and intelligent HVAC management. The simulation also compares the energy consumed by the AI-controlled building with a baseline system, allowing the effectiveness of the proposed intelligent agent to be evaluated.

8. Simulation

The simulation demonstrates the operation of the intelligent agent in a multi-room building. The system consists of four rooms: Living Room, Bedroom, Office, and Kitchen.

At each simulation step, the environment generates sensor readings such as:

- Room temperature
- Occupancy status
- Light level
- Outdoor temperature

The intelligent agent analyzes these readings and determines:

- Whether HVAC should be OFF, COOL, or HEAT
- The required lighting intensity
- The energy consumed during the simulation period

The system then compares its energy consumption against a baseline building, where HVAC and lighting operate without intelligent optimization.

9. Screenshots of Full Code

The following screenshots document the complete implementation of the AI Smart Energy Management System.

```
class Room:
    def __init__(self, name, comfort_temp=23.0, comfort_band=2.0):
        self.name = name
        self.temperature = round(random.uniform(20, 30), 1)
        self.occupied = False
        self.light_level = random.choice([0, 30, 60, 90]) # lux (0=dark)
        self.comfort_temp = comfort_temp
        self.comfort_band = comfort_band
        self.hvac_state = "OFF" # OFF / COOL / HEAT
        self.light_state = 0 # 0-100 %

    def sensor_reading(self):
        """Sensors are noisy -> partial observability."""
        noisy_temp = self.temperature + random.uniform(-0.3, 0.3)
        return {
            "room": self.name,
            "temp": round(noisy_temp, 1),
            "occupied": self.occupied,
            "light": self.light_level,
        }

class BuildingEnvironment:
    """Dynamic, stochastic, partially-observable, multi-agent (rooms act
    concurrently) environment representing a smart building."""

    def __init__(self, rooms):
        self.rooms = rooms
        self.outdoor_temp = 32.0
        self.hour = 6 # simulation starts at 06:00
```

Figure 2: Environment and Room Class

```
def step_dynamics(self):
    """Environment evolves on its own (dynamic) with random shocks
    (stochastic): occupancy changes, weather drifts, drafts, etc."""
    self.hour = (self.hour + 1) % 24
    # outdoor temperature follows a rough daily curve + random noise
    base = 26 + 8 * abs(12 - self.hour) / 12
    self.outdoor_temp = round(base + random.uniform(-1.5, 1.5), 1)

    for room in self.rooms:
        # stochastic occupancy
        room.occupied = random.random() < (0.75 if 8 <= self.hour <= 20 else 0.15)
        # natural heat leakage toward outdoor temperature
        drift = (self.outdoor_temp - room.temperature) * 0.08
        room.temperature = round(room.temperature + drift + random.uniform(-0.2, 0.2), 1)
        # HVAC effect from the PREVIOUS action
        if room.hvac_state == "COOL":
            room.temperature -= 1.1
        elif room.hvac_state == "HEAT":
            room.temperature += 1.1
        room.temperature = round(room.temperature, 1)
        # ambient daylight roughly follows time of day
        daylight = max(0, 100 - abs(13 - self.hour) * 12) + random.uniform(-5, 5)
        room.light_level = round(max(0, daylight), 1)
```

Figure 3: Environment Dynamics and Sensor Simulation

```

# -----
# 2. UTILITY-BASED AGENT
# -----
class SmartEnergyAgent:
    """
    Percept -> sensor_reading() from every room + outdoor temperature
    Action -> set HVAC state, set light dimmer %, toggle standby plugs
    Goal -> minimise kwh consumed
    Utility -> U = -energy_cost - comfort_penalty
    """

    HVAC_POWER = {"OFF": 0.0, "COOL": 1.5, "HEAT": 1.8} # kW
    LIGHT_MAX_POWER = 0.1 # kW at 100%
    STANDBY_SAVING = 0.05 # kW per idle device

    def __init__(self):
        self.total_energy_kwh = 0.0
        self.baseline_energy_kwh = 0.0
        self.comfort_violations = 0
        self.log = []

```

Figure 4: Smart Energy Management Agent

```

def decide_hvac(self, room):
    """Utility-based decision: only condition occupied, out-of-band rooms."""
    low = room.comfort_temp - room.comfort_band
    high = room.comfort_temp + room.comfort_band
    pre_low = room.comfort_temp - room.comfort_band + 0.6 # act pre-emptively
    pre_high = room.comfort_temp + room.comfort_band - 0.6
    if not room.occupied:
        return "OFF" # save energy when empty
    if room.temperature > pre_high:
        return "COOL"
    if room.temperature < pre_low:
        return "HEAT"
    return "OFF"

def decide_light(self, room):
    """Dim / switch off lights using daylight harvesting + occupancy."""
    if not room.occupied:
        return 0
    # daylight harvesting: less artificial light needed when bright outside
    needed = max(0, 70 - room.light_level)
    return int(min(100, needed))

```

Figure 5: Intelligent HVAC and Lighting Decision Functions

```

def act(self, env: BuildingEnvironment):
    step_energy = 0.0
    baseline_step_energy = 0.0
    readings = []

    for room in env.rooms:
        percept = room.sensor_reading()
        readings.append(percept)

        hvac_action = self.decide_hvac(room)
        light_action = self.decide_light(room)

        room.hvac_state = hvac_action
        room.light_state = light_action

        step_energy += self.HVAC_POWER[hvac_action]
        step_energy += self.LIGHT_MAX_POWER * (light_action / 100)

    # Baseline = "dumb" building: HVAC always ON to comfort temp,
    # lights always at 100% regardless of occupancy/daylight.
    baseline_step_energy += self.HVAC_POWER["COOL"]
    baseline_step_energy += self.LIGHT_MAX_POWER

```

Figure 6: Agent Actions and Energy Consumption Calculation

```

# comfort check
low = room.comfort_temp - room.comfort_band
high = room.comfort_temp + room.comfort_band
if room.occupied and not (low <= room.temperature <= high):
    self.comfort_violations += 1

self.total_energy_kwh += step_energy
self.baseline_energy_kwh += baseline_step_energy

self.log.append({
    "hour": env.hour,
    "outdoor": env.outdoor_temp,
    "energy_kwh": round(step_energy, 3),
    "baseline_kwh": round(baseline_step_energy, 3),
    "rooms": [(r.name, r.temperature, r.occupied, r.hvac_state, r.light_state)
               for r in env.rooms],
})
return readings

```

Figure 7: Comfort Evaluation and Performance Logging

```

# -----
# 3. SIMULATION LOOP
# -----
def run_simulation(hours=24):
    rooms = [Room("Living Room"), Room("Bedroom"), Room("Office"), Room("Kitchen")]
    env = BuildingEnvironment(rooms)
    agent = SmartEnergyAgent()

    print(f"{'Hr':>3} | {'Out(C)':>6} | " +
          " | ".join(f"{r.name[:10]:>10}" for r in rooms) + " | kWh(Agent) | kWh(Base)")
    print("-" * 100)

    for _ in range(hours):
        env.step_dynamics()
        agent.act(env)
        last = agent.log[-1]
        room_str = " | ".join(
            f"{t:>5.1f}{'*' if occ else ' '}{h[0]}" for (_, t, occ, h, l) in last["rooms"]
        )
        print(f"{last['hour']:>3} | {last['outdoor']:>6.1f} | {room_str} | "
              f"{last['energy_kwh']:>10.2f} | {last['baseline_kwh']:>9.2f}")

```

Figure 8: 24-Hour Simulation Loop

```

saved = agent.baseline_energy_kwh - agent.total_energy_kwh
saved_pct = 100 * saved / agent.baseline_energy_kwh

print("\n" + "=" * 60)
print("SIMULATION SUMMARY (24-hour cycle)")
print("=" * 60)
print(f"Agent total consumption      : {agent.total_energy_kwh:8.2f} kWh")
print(f"Baseline (no AI) consumption: {agent.baseline_energy_kwh:8.2f} kWh")
print(f"Energy saved                    : {saved:8.2f} kWh  ({saved_pct:5.1f}% reduction)")
print(f"Comfort violations              : {agent.comfort_violations} (out of "
      f"{hours * len(rooms)} room-hours)")
print(f"Comfort satisfaction rate : "
      f"{100 * (1 - agent.comfort_violations / (hours * len(rooms))):5.1f}%")
print("=" * 60)
return agent

```

Figure 9: Performance Evaluation and Final Output

10. Output

The final output of the simulation includes the total energy consumed by the AI agent, baseline energy consumption, energy saved, comfort violations, and comfort satisfaction rate.

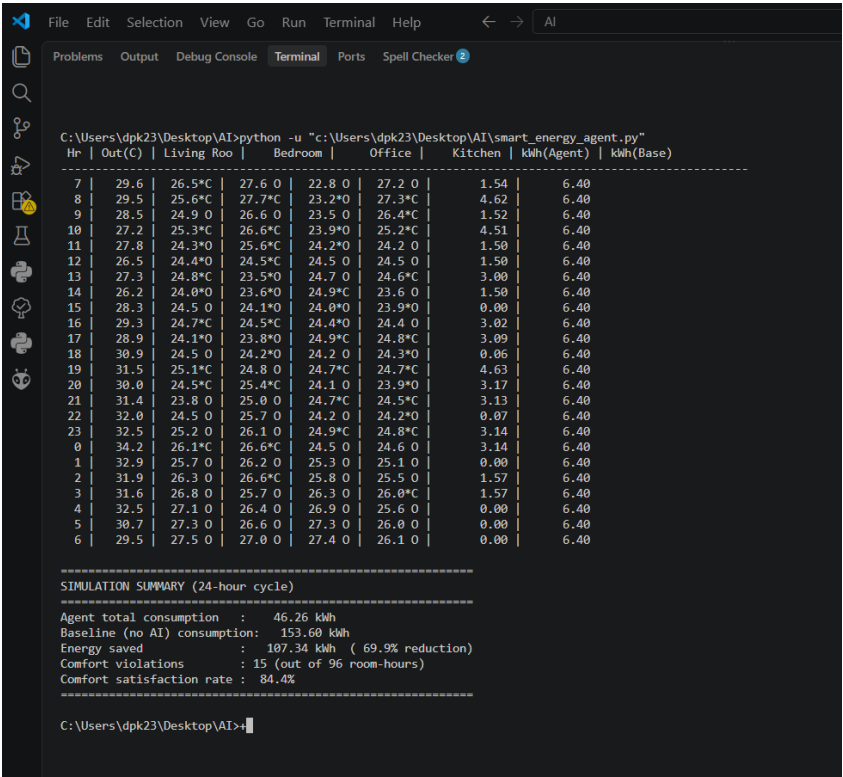


Figure 10: Program Execution